Demostrador automático de teoremas basado en resolución lineal.

Índice

Introducción **3**

Conceptos fundamentales: Forma clausal, resolución lineal **3**
Forma clausal 3
Resolución lineal 3

Implementación del demostrador automático de teoremas **4**
Representación de las formulas 4
Paso a forma clausal 5
Resolución lineal 6
Uniéndolo todo 7

Ejemplo de Ejecucion **7**

Conclusiones **7**

Bibliografía **8**

Autor: Sara de la Hoz Escuderos
Autor: Marcos Hurtado Morcillo
Autor: Marcos Galán Rodrigez

Introducción

El objetivo de este trabajo es implementar en Prolog un demostrador automático de teoremas para la lógica de proposiciones, utilizando la resolución lineal como método de prueba. Para ello, el sistema recibe una fórmula lógica, la transforma a forma clausal y aplica resolución lineal sobre el conjunto de cláusulas resultante hasta obtener la cláusula vacía o simplificar la fórmula todo lo posible.

La demostración automática de teoremas tiene como objetivo verificar de forma mecánica si una fórmula es consecuencia lógica de un conjunto de hipótesis. Los demostradores modernos operan, en su mayoría, por refutación: en lugar de probar directamente que una fórmula A es válida, niegan A e intentan derivar una contradicción. Si se llega a la cláusula vacía, queda demostrado que la fórmula original era cierta.

Prolog es un lenguaje especialmente adecuado para este tipo de tarea. Por un lado, permite definir operadores propios mediante el predicado `op/3`, lo que hace posible escribir fórmulas lógicas de forma natural usando símbolos como `~`, `/\`, `\/` y `=>`. Por otro lado, su mecanismo de unificación y su evaluación basada en búsqueda hacen que la manipulación simbólica de fórmulas sea directa y expresiva.

El trabajo se estructura en tres partes principales. Primero se explican los conceptos teóricos necesarios: la forma clausal y la resolución lineal. A continuación se describe la implementación en Prolog, detallando cada predicado desarrollado. Finalmente se presentan las conclusiones obtenidas.

Conceptos fundamentales: Forma clausal, resolución lineal

Forma clausal

La forma clausal es una manera estándar de representar una fórmula lógica que facilita la aplicación mecánica de reglas de inferencia. Una fórmula está en forma clausal cuando se expresa como un conjunto de cláusulas, donde cada cláusula es una disyunción de literales, y un literal es una variable proposicional o su negación.

Por ejemplo, la fórmula $(A \vee \neg B \vee C) \wedge (\neg A \vee D)$ se representa en forma clausal como el conjunto $\{\{A, \neg B, C\}, \{\neg A, D\}\}$.

La razón por la que la forma clausal ocupa un lugar central en la demostración automática es que un conjunto de cláusulas y la fórmula original que representa son equivalentes respecto a la insatisfacibilidad. Esto es precisamente lo que necesitamos para las pruebas por refutación: si conseguimos demostrar que el conjunto de cláusulas es insatisfacible (derivando la cláusula vacía), podemos concluir que la fórmula negada es insatisfacible y, por tanto, que la fórmula original es válida.

Resolución lineal

La resolución es una regla de inferencia introducida por J.A. Robinson en 1965 que opera sobre fórmulas en forma clausal. Dadas dos cláusulas que contienen un par de literales complementarios (L en una y $\neg L$ en la otra), la resolución elimina ese par y combina los literales restantes en una nueva cláusula llamada resolvente.

Una aplicación sin restricciones de la resolución puede generar un número enorme de cláusulas redundantes. Para reducir el espacio de búsqueda se utilizan estrategias de

resolución. La resolución lineal es una de ellas: impone que cada nuevo resolvente se obtenga a partir de la cláusula producida en el paso anterior (cláusula central), resuelta contra otra cláusula del conjunto original o contra un resolvente ya obtenido (cláusula lateral).

Formalmente, dado un conjunto de cláusulas Δ y una cláusula inicial $C_0 \in \Delta$, una deducción lineal es una secuencia $C_0, C_1, \ldots, C_n$ donde cada C_{i+1} es el resolvente de C_i con alguna cláusula lateral B_i, siendo $B_i \in \Delta$ o $B_i = C_j$ para algún $j < i$.

La resolución lineal es completa para la lógica de proposiciones: si el conjunto de cláusulas es insatisfacible, siempre existe una deducción lineal que llega a la cláusula vacía. Esta propiedad es la que garantiza que el demostrador implementado podrá encontrar la refutación siempre que exista.

En el contexto de este trabajo, la resolución lineal se aplica sobre el conjunto de cláusulas obtenido tras la transformación a forma clausal. La cláusula resultante tras aplicar todos los pasos de resolución es la solución que devuelve el sistema: si es la cláusula vacía, la fórmula original es una tautología (o el conjunto de hipótesis es insatisfacible).

Implementación del demostrador automático de teoremas

Representación de las formulas

Primero necesitamos representar las fórmulas de la lógica de proposiciones:

- Negación: usaremos el símbolo "`~`"
- Conjunción: usaremos el símbolo "`/\`"
- Disyunción: usaremos el símbolo "`\/`"
- Implicación: usaremos el símbolo "`=>`"

De esta forma podremos escribir formulas como: `p => r /\ s`

Si queremos trabajar con estas fórmulas en Prolog vamos a tener que definir estos operadores para que prolog los pueda interpretar. Para esto usaremos el predicado `op/3`

El predicado `op/3` funciona de la siguiente manera: `op(prioridad, tipo, nombre)`.

```
:- op(200, fy, ~).
:- op(500, xfy, /\).
:- op(600, xfy, \/).
:- op(700, xfy, =>).
```

Así quedarían los operadores definidos. Elegimos usar números altos para las prioridades porque es recomendado para evitar conflictos con otros operadores definidos por prolog.

De esta manera Prolog entiende los operadores. Lo podemos comprobar con `write_canonical(p => r /\ s)`. Que nos devuelve `=>(p,/\(r,s))`.

Prolog entiende las fórmulas de esta forma: `operador(termino1, termino2)`.

Paso a forma clausal

Para poder trabajar con las proposiciones en la resolución lineal necesitamos representarlas en forma clausal. Es una manera de escribir una fórmula lógica como un conjunto de cláusulas, donde cada cláusula es una disyunción de literales, que a su vez son una variable o su negación. Ejemplo:

$$(A \vee \neg B \vee C) \wedge (\neg A \vee D) \;==\; \big\{\{A, \neg B, C\}, \{\neg A, D\}\big\}$$

Para transformar Las fbf en formas Clausales seguimos lo siguientes pasos:

Quitar las implicaciones (=>):

Queremos trasformar cualquier forma [p=>q] en [$\sim p \vee q$].

```
rmImpl(P=>Q, ~P1 \/ Q1):- !, rmImpl(P,P1), rmImpl(Q,Q1).
rmImpl(P/\Q, P1 /\ Q1):- !, rmImpl(P,P1), rmImpl(Q,Q1).
rmImpl(P\/Q, P1 \/ Q1):- !, rmImpl(P,P1), rmImpl(Q,Q1).
rmImpl(~P, ~P1):- !, rmImpl(P,P1).
rmImpl(P, P).
```

Con la primera regla trasformamos las implicaciones y con las demás nos aseguramos de cubrir cualquier caso complejo con operaciones anidadas, como una conjunción o disyunción de implicaciones, etc...

Lo siguiente es usar las leyes de Morgan para simplificar las negaciones, es decir necesitamos que las negaciones solo afecten a un a variable y no a una conjunción o disyunción de estas.

```
simNeg(~(~P), P1):- !, simNeg(P, P1).
simNeg(~(P\/Q), P1 /\ Q1):- !, simNeg(~P, P1), simNeg(~Q, Q1). % De Morgan
simNeg(~(P/\Q), P1 \/ Q1):- !, simNeg(~P, P1), simNeg(~Q, Q1). % De Morgan
simNeg(P/\Q, P1 /\ Q1):- !, simNeg(P, P1), simNeg(Q, Q1).
simNeg(P\/Q, P1 \/ Q1):- !, simNeg(P, P1), simNeg(Q, Q1).
simNeg(P, P).
```

Nos encargamos de quitar dobles negaciones y usamos las leyes de De Morgan en las conjunciones y disyunciones. Las demás reglas se aseguran de cumplir casos compuestos.

Lo siguiente es distribuir OR sobre AND. De esta forma conseguimos una conjunción de disyunciones.

```
dist(P\/(Q/\R), (P1\/Q1) /\ (P2\/R1)):- !, dist(P,P1), dist(P,P2), dist(Q,Q1),
    dist(R,R1).
dist((P/\Q)\/R, (P1\/R1) /\ (Q1\/R2)):- !, dist(P,P1), dist(Q,Q1), dist(R,R1),
    dist(R,R2).
dist(P/\Q, P1/\Q1) :- !, dist(P,P1), dist(Q,Q1).
dist(P\/Q, P1\/Q1) :- !, dist(P,P1), dist(Q,Q1).
dist(P, P).
fnc(F, FNF) :-
    dist(F, F1),
    (F == F1 -> FNF = F1 ; fnc(F1, FNF)).
```

Llamaremos a esta fase con ayuda del predicado `fnc` para asegurarnos de aplicar el algoritmo a todas las capas de operadores de forma recursiva hasta que no se produzcan más cambios.

Finalmente nos libramos de la notacion de operadores clasicos para reprentarlo como listas de listas en la forma clausal.

```
clausulas(P/\Q, L) :- !,
    clausulas(P, L1),
    clausulas(Q, L2),
    append(L1, L2, L).
clausulas(P, [C]) :-
    literales(P, C).
literales(P\/Q, L) :- !,
    literales(P, L1),
    literales(Q, L2),
    append(L1, L2, L).
literales(P, [P]).
```

Juntando todo el proceso anterior podemos crear el siguiente predicado.

```
fbf_a_clausal(F, Clausulas) :-
    rmImpl(F, F1),
    simNeg(F1, F2),
    fnc(F2, F3),
    clausulas(F3, Clausulas).
```

Resolución lineal

Para distinguir si dos literales son complementarios, definimos el siguiente predicado:

```
complementario(~X, X) :- !.
complementario(X, ~X).
```

Para abordar la resolución con múltiples fórmulas, hemos dividido la implementación en dos predicados. El primero es un auxiliar al que se le pasan únicamente dos cláusulas y se encarga de eliminar los literales complementarios entre ellas:

```
resAux([], X, X).
resAux(X, [], X).
resAux([X], [~X], []).
resAux(Central, Clausula, Resolvente):-
    \+ (member(X, Central), member(X2, Clausula), complementario(X, X2)),
    append(Central, Clausula, Raux),
    list_to_set(Raux, Resolvente).
resAux(Central, Clausula, Resolvente):-
    member(X, Central),
    member(X2, Clausula),
    complementario(X, X2),
    delete(Central, X, L),
    delete(Clausula, X2, L2),
    resAux(L, L2, Resolvente).
```

Los dos primeros casos son los casos base para cláusulas vacías; el tercero detecta la contradicción devolviendo `[]`; el cuarto, usando negación por fallo (`\+`), une ambas cláusulas cuando no hay literales complementarios; y el quinto elimina recursivamente cada par complementario encontrado mediante `member`, `complementario` y `delete`.

El segundo predicado utiliza el auxiliar anterior para aplicar la resolución lineal partiendo de una cláusula central:

```
%resulucionLineal(Listadelistas, resultado).
resolucionLineal([], []).
resolucionLineal([[A]], [A]).
resolucionLineal([Clausula1, Clausula2], Resultado):-
    resAux(Clausula1, Clausula2, Resultado).
resolucionLineal([Cabeza, C1 |Cola], Resultado):-
    resAux(Cabeza, C1, Solucion1),
    append(Solucion1, Cola, NuevaLista),
    resolucionLineal(NuevaLista, Resultado).
```

Uniéndolo todo

Por último, el predicado principal que une todo el proceso: dado una fórmula en lógica de primer orden, la convierte a forma clausal y le aplica la resolución lineal para obtener la solución.

```
resolver(F, Solucion) :-
    fbf_a_clausal(F, C),
    resolucionLineal(C, Solucion).
```

Ejemplo de Ejecucion

```
?- resolver((p => q) /\ p /\ ~q, Solucion).
false.
```

El conjunto es insatisfacible (se deriva contradicción).

```
?- resolver((p => q) /\ p, Solucion).
Solucion = [q].
```

Conclusiones

Se ha implementado con éxito en Prolog un demostrador automático de teoremas para la lógica de proposiciones basado en resolución lineal. El sistema es capaz de recibir una fórmula escrita con los operadores `~`, `/\`, `\/` y `=>`, transformarla automáticamente a forma clausal y aplicar resolución lineal para obtener la cláusula resultante.

La implementación ha requerido resolver varios problemas concretos. El primero es la representación de las fórmulas: gracias al predicado `op/3` de Prolog, es posible definir los operadores lógicos con sus prioridades y asociatividades correctas, de forma que Prolog pueda parsear y manipular las fórmulas de modo simbólico.

El segundo problema es la transformación a forma clausal, que se ha descompuesto en cuatro predicados independientes y encadenados: `rmImpl` para eliminar implicaciones, `simNeg` para simplificar negaciones aplicando las leyes de De Morgan, `fnc`/`dist` para distribuir las disyunciones sobre las conjunciones de forma recursiva hasta estabilizar la fórmula, y finalmente `clausulas`/`literales` para convertir la FNC en una lista de listas. Al separarlo así, cada parte es más fácil de entender y probar por separado.

El tercer problema es la resolución lineal. La separación en dos predicados —`resAux`, que resuelve un par de cláusulas eliminando todos los literales complementarios, y `resolucionLineal`, que gestiona la secuencia lineal de resoluciones— hace que el algoritmo sea más claro y manejable. El predicado `complementario` es necesario para saber cuándo dos literales se cancelan entre sí.

Prolog ha resultado ser un lenguaje idóneo para este trabajo. Su modelo de ejecución por resolución SLD, su unificación incorporada y su soporte para operadores definidos por el usuario permiten expresar la lógica del demostrador de forma concisa y muy próxima a la especificación formal. Esto hace que el código resultante sea compacto y fácil de entender.

Como posibles mejoras futuras se podría extender el demostrador a la lógica de predicados de primer orden, lo que requeriría incorporar unificación general con variables, funciones de Skolem para eliminar cuantificadores existenciales y un mecanismo para renombrar variables entre cláusulas. También podría añadirse una traza del árbol de deducción que muestre los pasos de resolución aplicados, lo que facilitaría la verificación y comprensión de las pruebas generadas.

Bibliografía

"Programming in Prolog" by W.F.Clocksin & C.S.Mellish
JULIÁN IRANZO, P. *Programación Declarativa*. Universidad de Castilla-La Mancha, 2007.